Unit-3 - Question 1

Functions in Python

Question 1: Define functions. Explain the creation and calling of a function with suitable examples.

1) **Definition of function:**

A function is a reusable block of code that performs a specific task.

2) **Need / Importance of functions:**

- Code reuse - write once, use many times.
- Improves readability and organization of the program.
- Promotes modularity - large programs are divided into small parts.
- Easier testing and debugging.
- Reduces repetition and saves time.

3) **Syntax / Creation of a function:**

```
def function_name(parameters):
    statements
    return value    # optional
```

- def keyword: used to define a function.
- function_name: any valid identifier following naming rules.
- parameters: variables listed in parentheses to receive values.
- function body: the block of statements that performs the task.
- return statement: sends a value back to the caller (optional).

4) **Calling a function:**

A function is executed (runs) only when it is called.

Syntax: function_name(arguments)

5) **Types of simple functions (based on parameters and return value):**

- Function without arguments
- Function with arguments
- Function with return value

6) **Example 1 - Function without arguments (no return value):**

```
def greet():
    print("Hello")
greet()
```

Output:

Hello

7) **Example 2 - Function with arguments and return value:**

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

Output:

30

8) **Parameter idea:**

Parameters are variables listed in the function definition.
They receive values (arguments) at the time of function call.
These values are local to the function.

9) **Advantages / Summary:**

- Reusability
- Better structure
- Easy maintenance
- Faster development
- Reduces code length
- Improves teamwork in large projects

10) **Conclusion:**

Functions make programs short, organized and reusable.
They are a key concept for writing efficient and modular Python programs.

Unit-3 - Question 2

Different Function Categories

Question 2: Explain the different function categories with examples (like function with arguments with return statement).

1) **Introduction:**

Functions can be classified based on arguments (inputs) and return value (output).

2) **Category 1: Function without arguments and without return value.**

Example:

```
def show_msg():
    print("Welcome")
show_msg()
```

Output:

Welcome

3) **Category 2: Function with arguments and without return value.**

Example:

```
def greet(name):
    print("Hello", name)
greet("Riya")
```

Output:

Hello Riya

4) **Category 3: Function without arguments and with return value.**

Example:

```
def get_pi():
    return 3.14
print(get_pi())
```

Output:

3.14

5) **Category 4: Function with arguments and with return value.**

Example:

```
def add(a, b):
    return a + b
print(add(5, 7))
```

Output:

12

6) **Comparison note:**

Arguments are inputs given to a function. Return value is the output produced by the function.

7) **Use of each category:**

- Category 1: Used for actions or tasks that just perform a job (like displaying a message) and do not return any result.
- Category 2: Used when input is needed to perform a task, but no result needs to be returned.
- Category 3: Used when a function calculates or produces a value without needing any input.
- Category 4: Used when input is needed and a result must be returned for further use.

8) **Conclusion:**

Functions are powerful building blocks in Python. The choice of category depends on whether input is needed (arguments) and whether a result must be returned (return value).

Unit-3 - Question 3

Various Function Arguments

Question 3: List and explain various function arguments with suitable examples.

1) **Introduction:**
   Arguments are values passed to a function when it is called.

2) **Positional arguments:**
   Values are passed to parameters in correct position (order matters).

```
def sub(a, b):
    print(a - b)
sub(10, 3)
```

Output:
7

3) **Keyword arguments:**
   Values are passed by parameter name. Order does not matter.

```
def student(name, age):
    print(name, age)
student(age=20, name="Asha")
```

4) **Default arguments:**
   Function provides a default value if no argument is passed.

```
def greet(name="Guest"):
    print("Hello", name)
greet()
greet("Ram")
```

Output:
Hello Guest
Hello Ram

5) **Variable-length arguments (*args):**
   Accepts any number of positional arguments as a tuple.

```
def total(*nums):
    s = 0
    for x in nums:
        s += x
    print(s)
total(10, 20, 30)
```

Output:
60

6) **Keyword variable-length arguments (**kwargs):**
   Accepts any number of keyword arguments as a dictionary.

```
def info(**data):
    print(data)
info(name="Riya", age=21)
```

Output:
{'name': 'Riya', 'age': 21}

7) **Rules / Notes:**
   - Positional arguments must come first, then keyword arguments.
   - Default arguments should follow non-default parameters.

8) **Summary (Type and Use):**

Type of Argument	Use
• Positional arguments	Passed in order of position.
• Keyword arguments	Passed by parameter name.
• Default arguments	Parameter has a default value.
• *args (variable-length)	Accepts many positional arguments.
• **kwargs (keyword variable-length)	Accepts many keyword arguments.

9) **Conclusion:**
   Python supports different ways to pass arguments to functions, making code flexible, reusable and easy to use.

Unit-3 - Question 4

Can a Function Return Different Types of Data?

Question 4: Can a function returns multiple type of data? Justify with suitable example.

1) **Answer:**

Yes, a Python function can return values of different data types.

2) **Explanation:**

Python is dynamically typed, which means the return type of a function is not fixed. It depends on what the function returns.

3) **Examples – Returning different single data types:**

- Integer:

```
def get_num():
    return 10
```

→ returns integer

- String:

```
def get_name():
    return "Python"
```

→ returns string

- List:

```
def get_list():
    return [1, 2, 3]
```

→ returns list

- Dictionary:

```
def get_data():
    return {"name":"Riya", "age":20}
```

→ returns dictionary

4) **Returning multiple values in Python:**

```
def calc(a, b):
    return a+b, a-b

x, y = calc(10, 5)
print(x, y)
```

Output:

```
15 5
```

- Multiple values are returned as a tuple. In the example above, `calc(10, 5)` returns the tuple `(15, 5)`.

5) **Important point:**

A function returns only one object. That object may be of any type like integer, string, list, dictionary, tuple, etc.

6) **Practical uses:**

- Return multiple results from calculations (e.g., sum and difference).
- Return data and status together (e.g., data, error_code).
- Return structured information like dictionaries.
- Improves code readability and reduces the need for global variables.

7) **Conclusion:**

Yes, a Python function can return different types of data. It can return any single data type, and it can also return multiple values at once (as a tuple). This makes functions flexible and powerful in real-world programming.

Unit-3 - Question 5

Recursion in Python

Question 5: Define recursion? Explain with suitable program.

① **Definition:**

Recursion is a technique in which a function calls itself.

② **Base case and Recursive case:**

- Base case: A condition under which the function stops calling itself.
- Recursive case: The part where the function calls itself with a smaller or simpler input.

③ **Working steps of recursion (simple):**

- The function is called.
- It checks the base case.
- If base case is true, it returns a result.
- Otherwise, it calls itself with a smaller input.
- Results are returned back step by step until the first call gets the final answer.

④ **Advantages:**

- Short and clean code for some problems.
- Natural way to solve problems like factorial, tree traversal, Fibonacci.

⑤ **Disadvantages:**

- Uses more memory (function call stack).
- May be slower than iterative solution.
- Limited by recursion depth (maximum calls).

⑥ **Main program example:** Factorial using recursion.

```
def fact(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * fact(n-1)

print(fact(5))
```

Output:

```
120
```

⑦ **Tiny trace (how fact(5) works):**

- fact(5) = 5 * fact(4)
- fact(4) = 4 * fact(3)
- fact(3) = 3 * fact(2)
- fact(2) = 2 * fact(1)
- fact(1) = 1 (base case)

Then values return back:
fact(2) = 2, fact(3) = 6, fact(4) = 24, fact(5) = 120.

⑧ **Another common recursion example:**

Fibonacci sequence.

- fib(n) = fib(n-1) + fib(n-2) with base cases fib(0) = 0, fib(1) = 1.
  Used to generate Fibonacci numbers.

⑨ **Conclusion:**

Recursion is a powerful technique for breaking a problem into smaller subproblems, but it should be used wisely due to memory and speed limits.

Unit-3 - Question 6

Modules and Built-in Modules

Question 6: Define module? Explain different built-in modules with examples.

1) Definition of module:

- A module is a file containing Python code such as functions, variables and classes.

2) Importance of modules:

- Promotes code reuse.
- Helps in organization of code.
- Makes code easier to maintain and update.

3) Built-in modules with simple examples:

a) **math module:** Provides mathematical functions.

- sqrt(x) - square root
- pow(x, y) - x raised to power y
- pi - value of π

```
import math
print(math.sqrt(25))   # Output: 5.0
print(math.pow(2, 3))  # Output: 8.0
print(math.pi)         # Output: 3.1415926535
```

b) **random module:** Used to generate random numbers and make random selections.

- randint(a, b) - random integer in range [a, b]
- choice(seq) - random item from a sequence

```
import random
print(random.randint(1, 10))  # Output: any
print(random.choice(['red', 'green', 'blue']))
                              # Output: any one
```

c) **datetime module:** Used to work with date and time.

- Provides classes like date, time, datetime.

```
from datetime import date
print(date.today())
    # Output: 2024-05-15 (today's date)
```

d) **os module:** Used for interacting with the operating system, files and folders.

- getcwd() - get current working directory
- listdir(path) - list files in a directory

```
import os
print(os.getcwd())
 # Output: e.g. C:\Users\User\Documents
```

4) A few more built-in modules (brief):

- sys - provides system-specific parameters and functions.
- statistics - provides functions for calculating statistical values like mean, median, mode.

5) Importing modules:

- A module must be imported before it can be used in a program.
  Syntax: `import module_name` or `from module_name import item(s)`

6) Advantages / Uses of built-in modules:

- Save time and effort by using ready-made functions.
- Increase program reliability and reduce errors.
- Provide powerful features for common tasks (math, files, date/time, etc.).
- Help in writing clean, short and efficient code.

7) Summary:

- A module is a file that contains Python definitions and statements.
- Python provides many built-in modules such as math, random, datetime, os, sys, statistics, etc.
- By importing and using these modules, we can perform complex tasks easily and write better Python programs.

Unit-3 - Question 7

import, from and as Keywords

Question 7: Explain import, from and as keywords with program.

1) **Brief introduction:**

The import, from and as keywords are used to bring module contents into a program so that we can use functions, classes or variables defined in that module.

2) **import keyword:**

- Syntax:

```
import module_name
```

- Example:

```
import math
print(math.sqrt(16))
```

→ Output: 4.0

3) **from keyword:**

- Syntax:

```
from module_name import item_name
```

- Example:

```
from math import sqrt
print(sqrt(49))
```

→ Output: 7.0

4) **as keyword (alias name):**

- Syntax:

```
import module_name as alias
```

- Example:

```
import math as m
print(m.pi)
```

→ Output: 3.141592653589793

5) **Combined example (using from and as together):**

```
from math import factorial as fact
print(fact(5))
```

→ Output: 120

6) **Differences between import, from and as:**

- **import**: imports the entire module. We access members using module_name.member.
- **from**: imports specific item(s) from a module. We can use them directly.
- **as**: gives an alias (short name) to a module or item for easier reference.

7) **Advantages:**

- Makes the code shorter and easier to write.
- Improves readability and clarity.
- Allows selective import to avoid bringing unnecessary names.

8) **Conclusion:**

The import, from and as keywords help in reusing code from modules efficiently. They support modular programming and make Python programs clean, organized and easier to maintain.

Unit-3 - Question 8

How to Create a Package

Question 8: How to create the package? Explain with an example.

1) **Definition:**

A package is a collection of related modules stored in a folder.

2) **Important Point:**

A package usually contains an __init__.py file.

3) **Steps to create a package:**

- Create a folder with package name.
- Add __init__.py file in that folder.
- Create module files (.py) inside the folder.
- Import and use the package in another file.

4) **Example - Package Structure:**

```
mypack/          ⟶ Package folder
  ├── __init__.py  ⟶ Makes it a package
  └── calc.py      ⟶ Module file
```

5) **Example - calc.py (module file):**

```
def add(a, b):
    return a + b
```

6) **Using the package in another file:**

```
from mypack.calc import add
print(add(10, 20))
```

Output: 30

7) **Benefits of using a package:**

- Better organization of code.
- Promotes modular design.
- Easy reuse of code in different projects.

8) **Package vs Module:**

- Module: A single Python file (.py) containing functions, classes, etc.
- Package: A folder containing multiple modules and an __init__.py file.

9) **Conclusion:**

Packages help in organizing related modules together, making the code clean, reusable, and easy to maintain.

Unit-3 - Question 9

Important Programs

Question 9: Programs

Important Unit-3 programs with simple code

1) Function to add two numbers.

```
def add(a, b):
    return a + b
print(add(7, 5))
```

Output: 12

2) Function to check even or odd.

```
def even_odd(n):
    if n % 2 == 0:
        return "Even"
    else:
        return "Odd"
print(even_odd(8))
```

Output: Even

3) Function with default argument greeting.

```
def greet(name="Guest"):
    return "Hello, " + name
print(greet())
print(greet("Alice"))
```

Output:

```
Hello, Guest
Hello, Alice
```

4) Function using keyword arguments.

```
def show(a, b, c):
    return a*100 + b*10 + c
print(show(c=3, a=1, b=2))
```

Output: 123

5) Function using *args to find sum.

```
def sum_all(*args):
    total = 0
    for num in args:
        total += num
    return total
print(sum_all(1, 2, 3, 4, 5))
```

Output: 15

6) Recursive factorial.

```
def fact(n):
    if n == 0:
        return 1
    else:
        return n * fact(n-1)
print(fact(5))
```

Output: 120

7) Recursive Fibonacci for n terms.

```
def fib(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    else:
        seq = fib(n-1)
        seq.append(seq[-1] + seq[-2])
        return seq
print(fib(6))
```

Output: [0, 1, 1, 2, 3, 5]

8) Recursive sum of digits.

```
def sum_digits(n):
    if n == 0:
        return 0
    return n % 10 + sum_digits(n // 10)
print(sum_digits(12345))
```

Output: 15

9) Program using math module to find square root.

```
import math
x = 25
print(math.sqrt(x))
```

Output: 5.0

10) Program using random module to print a random number.

```
import random
print(random.randint(1, 100))
```

Output: (any number between 1 and 100)

11) Program using alias import.

```
import math as m
print(m.pi)
```

Output: 3.141592653589793

12) Program showing package / custom module style usage.

```
from mypack.calc import add
print(add(5, 6))
```

Output: 11 (assuming mypack.calc.add returns the sum)